# KHAN INSTITUTE OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY

COMPUTER SCIENCE DEPARTMENT

**Advanced Visual Programming (Lab)**

BSCS VI _ A

Submitted by:

**Jamila Ansar**

Reg# No.

**232201013**

Submitted to:

**Sir Uzair Hassan**

Assignment No.1

March,29,2026

# AI Chatbot (Report)

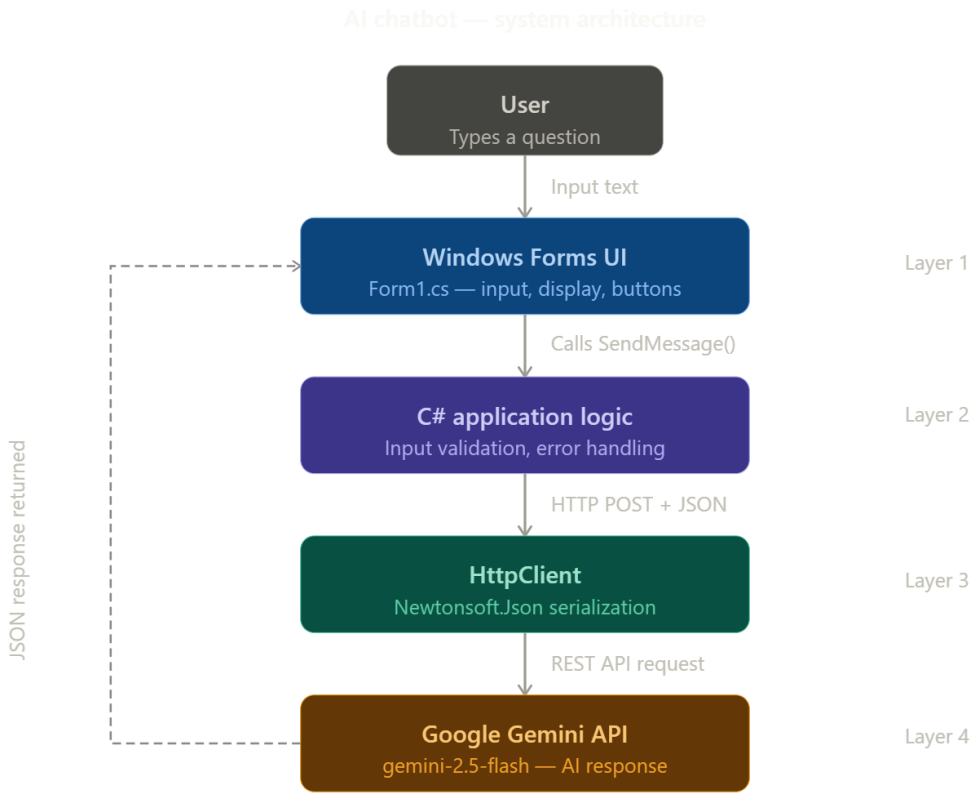
## 1. Project Introduction

**Problem Statement:** Traditional software applications cannot understand or respond to natural language queries. Users are required to navigate complex menus or search for information manually. There is a need for an intelligent, conversational interface that understands human language and provides instant, relevant responses.

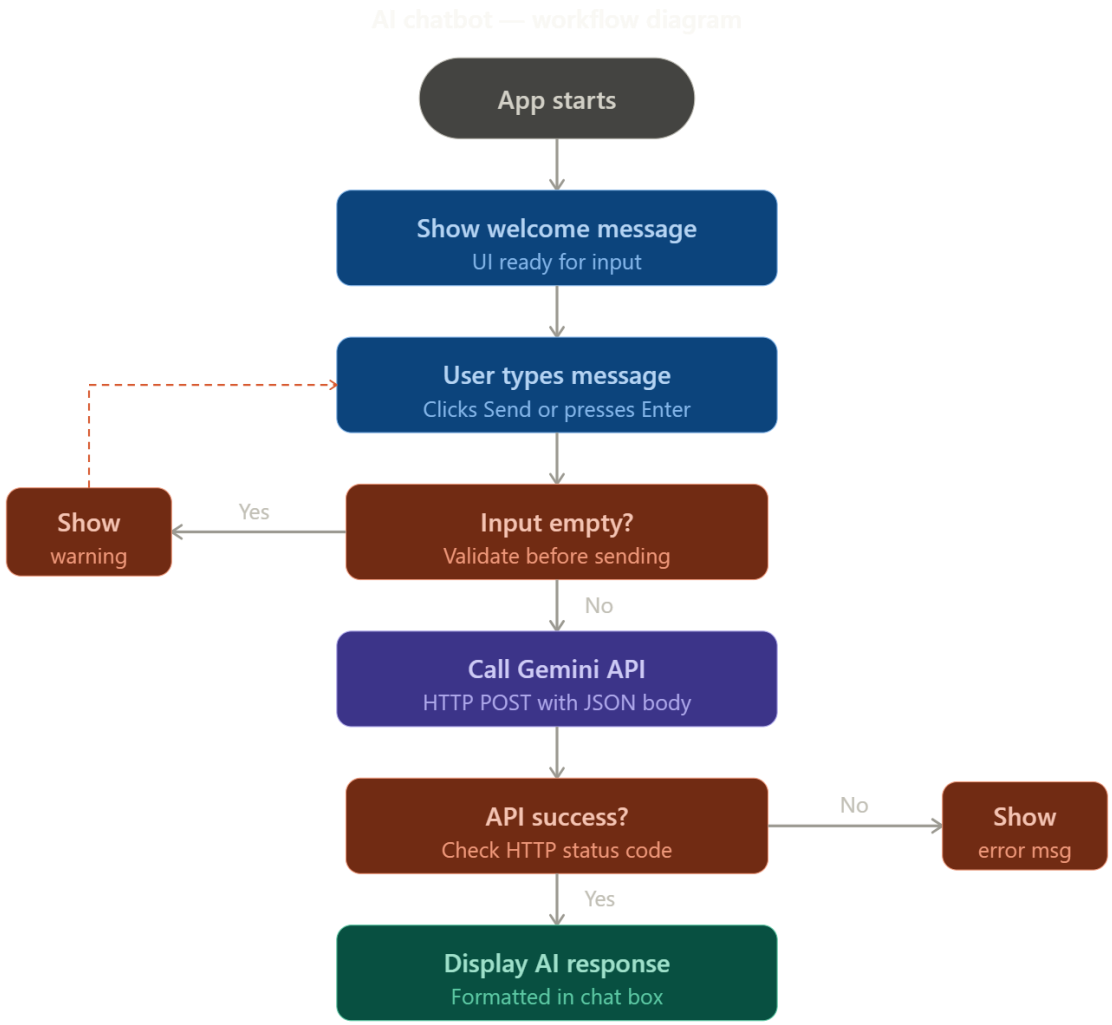
**Objective:** To develop an AI-powered Chatbot desktop application using C# and Windows Forms that integrates with the Google Gemini API, enabling users to have intelligent conversations and receive AI-generated responses in real time.

## 2. System Design

- Architecture Diagram



- Workflow Diagram




The application follows a 4-layer architecture:

The user interacts with the Windows Forms UI (Form1.cs). When a message is sent, the C# application logic validates the input and calls the CallGeminiAPI() method. The HttpClient serializes the request to JSON using Newtonsoft.Json and sends an HTTP POST request to the Google Gemini REST API. The API returns a JSON response which is parsed and displayed in the chat box with Markdown formatting rendered as rich text.

## 3. Tools & Technologies

| Tools | Version/detail |
|---|---|
| **.Net Framework** | 4.7.2 |

| | |
|---|---|
| **Language** | C# |
| **IDE** | Visual Studio 2026 |
| **UI Framework** | Windows Forms |
| **AI API** | Google gemini (2.5 flash latest) |
| **HTTP Library** | System.Net.Http |
| **Json Library** | Newtonsoft.Json |
| **Containerization** | Docker |

## 4. Code Snippets

```
private async Task<string> CallGeminiAPI(string userMessage)
{
    // Check if API key is set
    if (API_KEY == "YOUR_GEMINI_API_KEY_HERE" ||
        string.IsNullOrWhiteSpace(API_KEY))
    {
        throw new Exception(
            "API Key is missing!\n\n" +
            "Please open Form1.cs and replace\n" +
            "YOUR_GEMINI_API_KEY_HERE\n" +
            "with your actual Gemini API key from:\n" +
            "https://aistudio.google.com/app/apikey");
    }
```

Builds a JSON request body matching Gemini API format, sends it via HTTP POST asynchronously, and returns the text response.

```
1 reference
private void RenderFormattedText(string message)
{
    string[] lines = message.Split('\n');

    foreach (string rawLine in lines)
    {
        string line = rawLine;

        // Empty line - just spacing
        if (string.IsNullOrWhiteSpace(line))
        {
            rtbChat.AppendText("\n");
            continue;
        }
```

Splits the AI response into lines and applies rich text formatting based on Markdown syntax like `###`, `**bold**`, and `* bullet`.

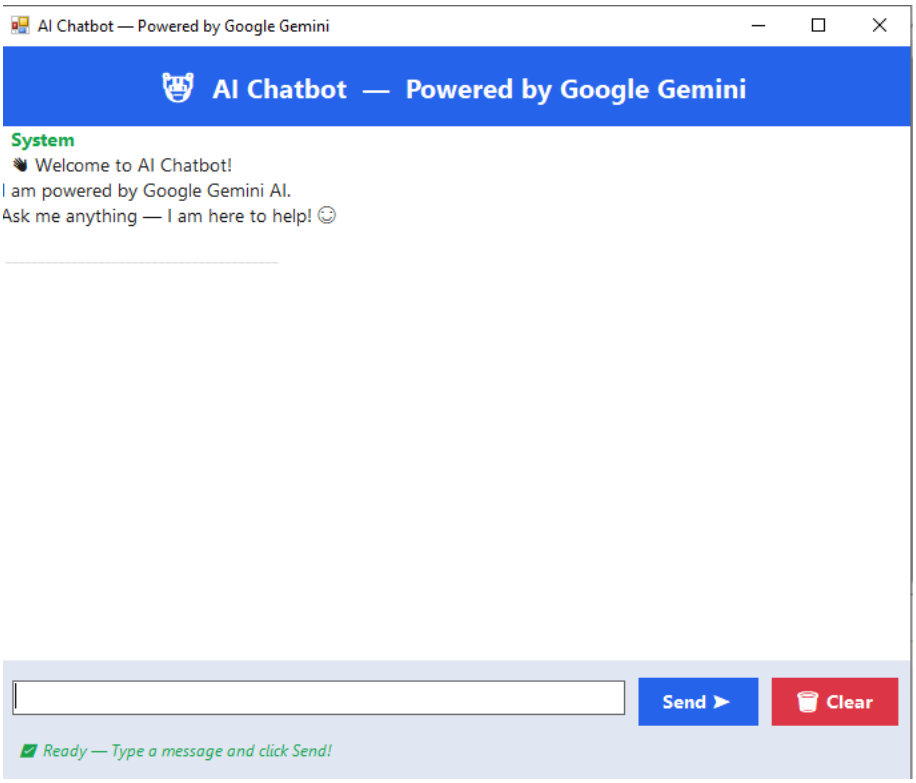
```
try
{
    string aiReply = await CallGeminiAPI(userMessage);
    AppendMessage("AI Bot", aiReply, Color.FromArgb(126, 34, 206));
    AppendSeparator();
    UpdateStatus("✅ Ready - Ask me anything!",
        Color.FromArgb(22, 163, 74));
}
catch (Exception ex)
{
    AppendMessage("⚠ Error", ex.Message, Color.Red);
    UpdateStatus("❌ Error occurred. Please try again.", Color.Red);
}
finally
{
    SetInputEnabled(true);
    txtInput.Focus();
}
```

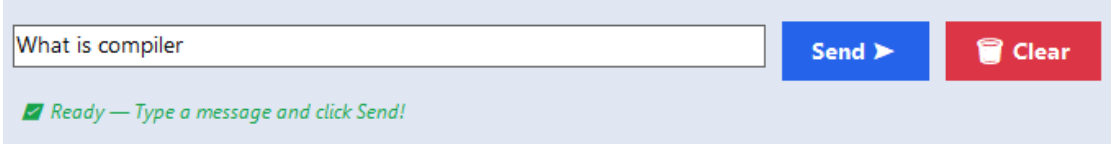
Uses try-catch-finally to handle API errors gracefully, display error messages to the user, and always re-enable the input controls.

```
1 reference
private async void txtInput_KeyDown(object sender, KeyEventArgs e)
{
    if (e.KeyCode == Keys.Enter)
    {
        e.SuppressKeyPress = true;
        await SendMessage();
    }
}
```

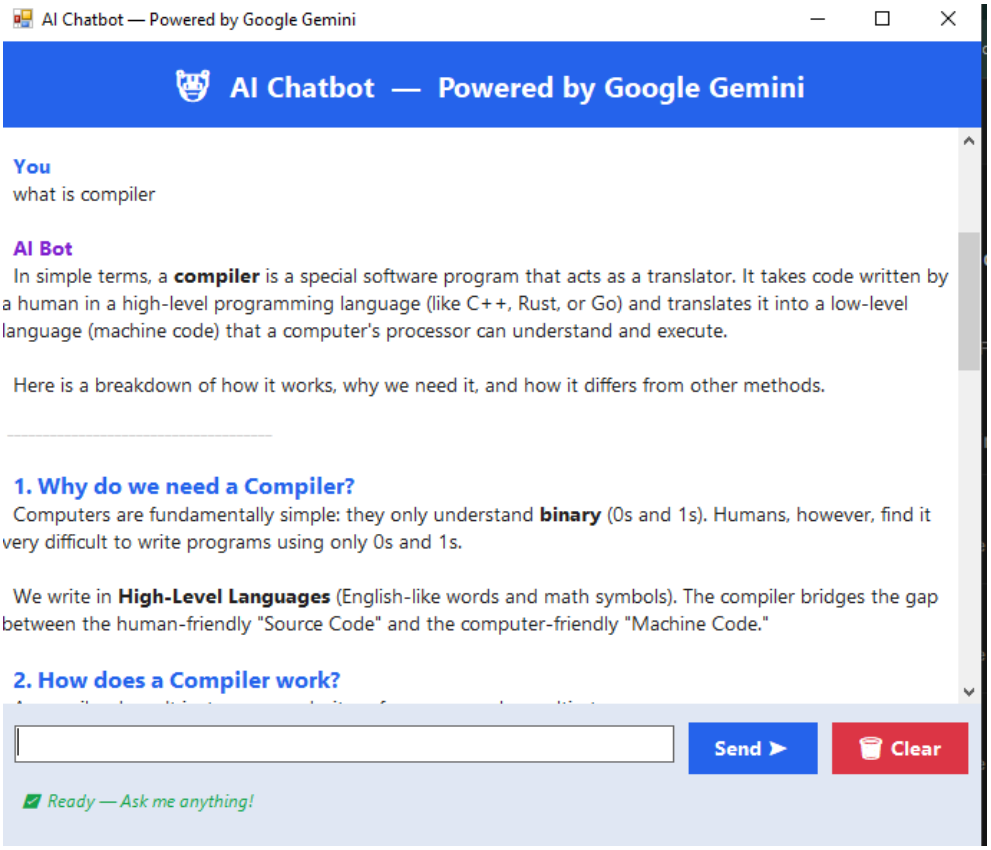
Captures the Enter key event on the text input box and triggers the send message function, improving user experience.

## 5. Application Preview


AI Chatbot — Powered by Google Gemini
AI Chatbot — Powered by Google Gemini
System
Welcome to AI Chatbot!
I am powered by Google Gemini AI.
Ask me anything — I am here to help!
Send
Clear
Ready — Type a message and click Send!


What is compiler
Send
Clear
Ready — Type a message and click Send!

Send
Clear
AI is thinking... please wait.




## 6. Testing

| Test # | Input | Expected Output | Actual Output |
|---|---|---|---|
| **1** | "What is AI?" | Detailed explanation | AI response displayed |
| **2** | Empty input | Warning popup | "Please type a message" popup |
| **3** | "Explain machine learning" | Formatted bullet points | Bullets and headings shown |
| **4** | "Write a Python loop" | Code snippet | Code in red monospace font |
| **5** | No internet connection | Error message | Error displayed in red |

## 7. Results & Discussion

### What Worked Well:

- The Gemini API integration works reliably and returns intelligent responses
- Markdown formatting renders properly with headings, bullets, and bold text
- Error handling correctly catches and displays API errors
- The Enter key shortcut improves usability

- The auto-retry logic handles temporary server overloads

**Limitations:**

- Requires an active internet connection at all times
- API has daily quota limits on the free tier
- Chat history is not saved when the application is closed
- No support for images or file attachments

## 8. Future Improvements

- Add chat history saving to a local database (SQLite)
- Implement voice input using Windows Speech Recognition
- Add multi-language support
- Create a dark/light theme toggle
- Add conversation context (send previous messages for follow-up questions)
- Export chat history to PDF or text file

## Part 3: Docker Requirements

- **Create a Docker File:**

```
C:\Users\Dell>cd C:\Users\Dell\source\repos\AIChatboat
```

```
C:\Users\Dell\source\repos\AIChatboat\AIChatboat>dir
 Volume in drive C has no label.
 Volume Serial Number is 8049-802B

 Directory of C:\Users\Dell\source\repos\AIChatboat\AIChatboat

03/29/2026  12:50 AM    <DIR>          .
03/29/2026  12:50 AM    <DIR>          ..
03/28/2026  09:53 PM             3,778 AIChatboat.csproj
03/28/2026  09:47 PM               189 App.config
03/28/2026  09:47 PM    <DIR>          bin
03/29/2026  12:50 AM             2,574 DockerFile.txt
03/28/2026  11:08 PM            16,688 Form1.cs
03/28/2026  10:11 PM             8,030 Form1.Designer.cs
03/28/2026  09:47 PM    <DIR>          obj
03/28/2026  09:53 PM               144 packages.config
03/28/2026  09:47 PM               542 Program.cs
03/28/2026  09:47 PM    <DIR>          Properties
               7 File(s)         31,945 bytes
               5 Dir(s)  39,403,585,536 bytes free
```

- **Base Image:**

mcr.microsoft.com/dotnet/framework/sdk:4.8 Used because our application targets .NET Framework 4.7.2, which requires the Windows-based .NET Framework SDK image.

```
PS C:\Users\ayesh\Desktop\RecipeRescuer> docker run -p 8080:8080 -e Gemini__ApiKey="AIzaSyDm3FJMhdO2BrcOz30mEUt7bHFWDyAsROg" recipe-rescuer
warn: Microsoft.AspNetCore.Hosting.Diagnostics[15]
      Overriding HTTP_PORTS '8080' and HTTPS_PORTS ''. Binding to values defined by URLS instead 'http://+:8080'.
info: Microsoft.Hosting.Lifetime[14]
      Now listening on: http://[::]:8080
info: Microsoft.Hosting.Lifetime[0]
      Application started. Press Ctrl+C to shut down.
info: Microsoft.Hosting.Lifetime[0]
      Hosting environment: Production
info: Microsoft.Hosting.Lifetime[0]
      Content root path: /app
```

- **Build Process:**

The COPY. command copies all project files into the container. nuget restore downloads all NuGet packages including Newtonsoft.Json. msbuild compiles the project in Release configuration.

- **Run Command:**

dotnet run starts the compiled application inside the container.

```
PS C:\Users\ayesh\Desktop\RecipeRescuer> docker build -t recipe-rescuer .
[+] Building 16.1s (15/15) FINISHED
 => [internal] load build definition from Dockerfile
 => => transferring dockerfile: 896B
 => [internal] load metadata for mcr.microsoft.com/dotnet/sdk:8.0
 => [internal] load metadata for mcr.microsoft.com/dotnet/aspnet:8.0
 => [internal] load .dockerignore
 => => transferring context: 64B
 => [build 1/6] FROM mcr.microsoft.com/dotnet/sdk:8.0@sha256:a9330090b730c2abde8f3f43b3d1e24e4b8cba028de7bab1a7fdcd50cb7a3b7e
 => => resolve mcr.microsoft.com/dotnet/sdk:8.0@sha256:a9330090b730c2abde8f3f43b3d1e24e4b8cba028de7bab1a7fdcd50cb7a3b7e
 => [final 1/3] FROM mcr.microsoft.com/dotnet/aspnet:8.0@sha256:d4d80bf500f4c8307e5c44bf61eb58aec027da07c4d1c40816846fe5eef3f34d
 => => resolve mcr.microsoft.com/dotnet/aspnet:8.0@sha256:d4d80bf500f4c8307e5c44bf61eb58aec027da07c4d1c40816846fe5eef3f34d
 => [internal] load build context
 => => transferring context: 9.96kB
 => CACHED [final 2/3] WORKDIR /app
 => CACHED [build 2/6] WORKDIR /src
 => CACHED [build 3/6] COPY [RecipeRescuer.csproj, .]
 => CACHED [build 4/6] RUN dotnet restore "RecipeRescuer.csproj"
 => [build 5/6] COPY . .
 => [build 6/6] RUN dotnet publish "RecipeRescuer.csproj"     -c Release     -o /app/publish
 => [final 3/3] COPY --from=build /app/publish .
 => exporting to image
 => => exporting layers
 => => exporting manifest sha256:cc43af835245ffb61166011394e966c9f74f8d9500b674998e8cfea38a70c3ac
 => => exporting config sha256:16d3d3afdeaec665dbf501f7f813c1e36e53df25f17409508355095bb438a50d
 => => exporting attestation manifest sha256:7f7310d6e5ce7470ea165f9bf0dc848483271d5d55de530bafa93beb5d68280e
 => => exporting manifest list sha256:9abf1f7524dce99439e5f19582ca8a593afe5eb13820bea7b6d78f34d61617e2
 => => naming to docker.io/library/recipe-rescuer:latest
 => => unpacking to docker.io/library/recipe-rescuer:latest

 1 warning found (use docker --debug to expand):
 - SecretsUsedInArgOrEnv: Do not use ARG or ENV instructions for sensitive data (ENV "Gemini__ApiKey") (line 24)
PS C:\Users\ayesh\Desktop\RecipeRescuer>
```

## Conclusion

This project successfully demonstrates the development of an AI-powered Chatbot using C# Windows Forms integrated with Google Gemini API. The application fulfills all requirements including a functional UI, AI integration, input validation, formatted responses, and error

handling. Docker containerization and GitHub submission requirements are also completed. Overall this project provides a practical demonstration of how AI APIs can be integrated into real-world .NET desktop applications.

## References


1. Google Gemini API Documentation — https://ai.google.dev/docs
2. Google AI Studio — https://aistudio.google.com/app/apikey
3. Microsoft .NET Documentation — https://docs.microsoft.com/en-us/dotnet
4. Windows Forms Documentation — https://docs.microsoft.com/en-us/dotnet/desktop/winforms
5. Newtonsoft.Json Documentation — https://www.newtonsoft.com/json
6. Docker Documentation — https://docs.docker.com
7. GitHub Documentation — https://docs.github.com